

AUTO ENCODER

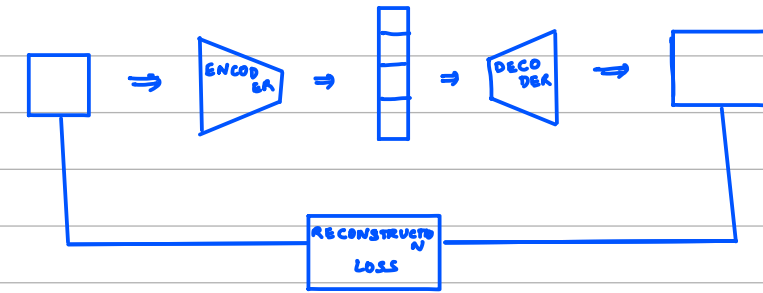


Image $\xrightarrow{\text{Encoder}}$ Lower dimension representation $\xrightarrow{\text{Decoder}}$ Image

- ↳ input $\rightarrow$ lower dimension input for other tasks
- ↳ Representation learning $\rightarrow$ Fill in images
- ↳ Denoising $\rightarrow$ Denoise a image.

LATENT SPACE

↳ Lower dimensional data point

Why VAE? and not AE.

- ↳ Push training points away from each in latent space.
 - $\Rightarrow$ Not all latent space points can generate a valid output
 - $\Rightarrow$ Noised input $\rightarrow$ noised output
- ↳ No new variation of data $\rightarrow$ no knowledge of dist. of data pts
 - $\Rightarrow$ cannot generate data it has't seen $\rightarrow$ memorised.

VALID IMAGES FROM ANY LATENT POINT:

Instead, encoding images to a point in latent space

↳ dist. of points in latent space.

↳ hope: allow to capture entire latent region.
Not exactly input but similar to input $\rightarrow$ Make network capable of generating variations of input.

CONSTRAINING DISTRIBUTION:

- $\Rightarrow$ Should not collapse encoder dist. to Singular point as we need wide distributions.
to capture entire latent space.
- $\Rightarrow$ Should disallow dist. to move far apart from one another.

↓
Soln: Standard Gaussian

- ↳ keep distri closer to each other.
- ↳ Ensure that distri have higher variances
- ↳ Hence disallow network to collapse dist. to a pt.
- ↳ Const. KL divergence b/w distri & Std. Gaussian

KL Divergence :

$$D_{KL}(P||Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

$$D_{KL}(P||Q) = \int P(x) \log \frac{P(x)}{Q(x)} dx.$$

Coin 1 :

$$P(H) = 0.5$$

$$P(T) = 0.5$$

Coin 2 :

$$P(H) = p$$

$$P(T) = q.$$

⇒ Easy to diff. if they are far apart.

⇒ Quantise the distance b/w the distrib.

$$\frac{P(\text{Observation} | \text{real coin})}{P(\text{Observation} | \text{coin 2})}$$

$$P(\text{Observation} | \text{real coin}) = p_1^{N_H} p_2^{N_T}$$

$$P(\text{Observation} | \text{coin 2}) = q_1^{N_H} q_2^{N_T}$$

$$\Rightarrow \log \left(\frac{p_1^{N_H} p_2^{N_T}}{q_1^{N_H} q_2^{N_T}} \right)^{1/N}$$

$$= \frac{N_H}{N} \log p_1 + \frac{N_T}{N} \log p_2 - \frac{N_H}{N} \log q_1 - \frac{N_T}{N} \log q_2$$

$$= p_1 \log \left(\frac{p_1}{q_1} \right) + p_2 \log \left(\frac{p_2}{q_2} \right) \quad \frac{N_H}{N} = p_1 \geq \frac{N_T}{N} = p_2.$$

⇒ Natural measurement of probability distrib. motivated by how likely the 2nd dist. will be able to generate from first distrib.
Cross entropy loss → min KL div ⇒ reducing dist. b/w distributions.

MODELING DATA GENERATION

⊗

→ Sample a point in latent space from a prior → $p(z)$

→ Generate a data point by sampling x from the distribution - $p(x|z)$.

⊗

Assume: $p(z)$ → Standard gaussian

↓
More math
Simple. $p(x|z)$ → gaussian whose mean is defined by $\mu: \mu$.

& CI Covariance matrix.

↑
aprox.
through decoder

$$p(x|z) = \mathcal{N}(\mu(z), CI)$$

Our set of images x , does not give us any info regarding hidden variable

$$p(z|x) = \frac{p(x|z)p(z)}{\sum_z p(x|z)p(z)} = \frac{p(x|z)p(z)}{\sum_z p(x|z)p(z)}$$

higher z dim

↳ cannot compute in polynomial time.

$$p(z|x) \sim q(z|x) = \mathcal{N}(\mu, \sigma^2).$$

Smart stuff:

Instead of getting exact cause of lot of time, we can div minimise, to get $q(z|x)$ that is close enough to $p(z|x)$.

$$\min KL(q(z|x), p(z|x))$$

$$\Rightarrow \sum q(z|x) \log \frac{q(z|x)}{p(z|x)}$$

$$\sum q(z|x) \log q(z|x) - \sum q(z|x) \log \frac{p(x|z)p(z)}{p(x)}$$

$$\Rightarrow \sum q(z|x) \log q(z|x) - \sum q(z|x) \log p(x|z) - \sum q(z|x) \log p(z) + \sum q(z|x) \log p(x)$$

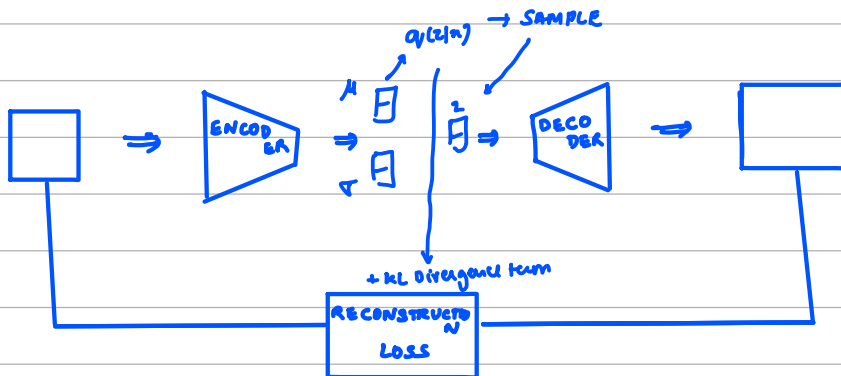
Const. ignored.

$$\Rightarrow \min KL(q(z|x), p(z|x)) - \sum q(z|x) \log p(x|z)$$

$$\min KL(q(z|x), p(z|x)) - E_{z \sim q(z|x)} \log p(x|z)$$

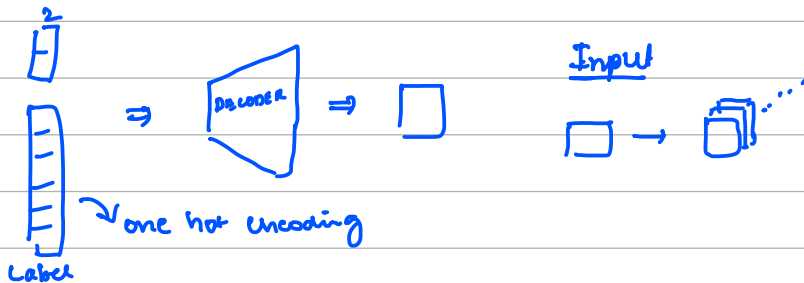
$$\min KL(q(z|x), p(z|x)) + E_{z \sim q(z|x)} \frac{1}{2c} \|x - \mu(z)\|^2$$

$\downarrow$ KL divergence loss $\downarrow$ Reconstm. Loss



No control over generated data. (Conditional VAE).

↳ So how to include class labels



IMPLEMENTATION:

⇒ Predict log (variance) instead of variance
↳ Stable.

⇒ Reparametrization trick.

↳ $q(z|x)$ Sampling → Stochastic
↓
backprop $d/d\theta$ → Deterministic

Thus, Sample from $N(0,1)$.
 $\tilde{z} = \tilde{\sigma} + \tilde{\mu}$
Deterministic

Auto encoder $\rightarrow$ $\square$ $\rightarrow$ Encoder $\rightarrow$ Latent Space $\rightarrow$ Decoder $\rightarrow$ $\square$

Variational Autoencoders $\rightarrow$ $\square$ $\rightarrow$ Encoder $\rightarrow \begin{matrix} \mu \\ \sigma \end{matrix} \rightarrow q(z|x) \rightarrow \text{Sample} \rightarrow z \rightarrow \text{Decoder} \rightarrow \square_x$

classes $\rightarrow$ $\begin{matrix} \square \\ \square \\ \square \end{matrix}$ $\rightarrow$ Encoder $\rightarrow \begin{matrix} \mu \\ \sigma \end{matrix} \rightarrow q(z|x, y) \rightarrow \text{Sample} \rightarrow z, h \rightarrow \text{Decoder} \rightarrow \square_x$

VQVAE:

$\rightarrow$ Complex data $\rightarrow$ low dimensional Latent space $\rightarrow$ weak/noisy $\rightarrow$ generate images like the mean of others.

make it big 😊 $\rightarrow$ High dimensional space $\rightarrow$ too many points very difficult to learn from.

Real world data not continuous $\rightarrow$ Interpolating b/w two classes does not make sense.

$\downarrow$ is discrete
sparsity.

Encoder $\Rightarrow$ Input: Feature map

Embedding space $\rightarrow \begin{matrix} \square \\ \square \\ \square \end{matrix} \dots K \times D$

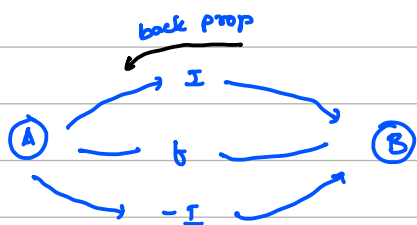
$H \times W \times D$ $\leftarrow$ Different Dimensionality.
 1×1 convolution.

$(grad \rightarrow zero)$
 $H \times W \times D$ $\rightarrow$ Find nearest embedding space vector $\leftarrow$ For each D Dimensional vector

$\downarrow$ $\rightarrow$ Decoder $\rightarrow$ output.

Replaced Embedding Space Vectors

① $\rightarrow f \rightarrow$ ② $B = f(A)$



$B = A + (f(A) - A) \cdot \text{detach}()$
So cheat system into making f differentiable.

codebook loss $\|S_g[z_e(x)] - c\|_2^2 \rightarrow$ Freeze Samples $\rightarrow$ Move embedding vectors to the sample.
(embedding space)

commitment loss $\|z_e(x) - S_g[c]\|_2^2 \rightarrow$ Freeze embedding vectors $\rightarrow$ Move samples towards embed vectors

$\beta \rightarrow$ emphasises the weight between these losses

KL Loss $\rightarrow$ ignored $\rightarrow P(z) = 1/K$

$z \rightarrow$ one hot vector

$\log K \rightarrow \text{const} \rightarrow$ thus ignored.